

Лабораторная работа

По дисциплине

«Теория функции комплексной переменной»

3 семестр

Выполнили:

Фан Нгок Туан

Кива Глеб Владимирович

Ибрагимова Айгуль Ильгизовна

Преподаватель:

Селеменчук Антон Сергеевич

Задание 1.....	3
Задание 2.....	4
1. Введение в множество Мандельброта.....	4
2. Алгоритм построения.....	4
3. Код программы.....	5
4. Изображение множества при различных максимальных количествах итераций.....	6
Задание 3.....	8
Код программы.....	8
Полученные изображения.....	9
Эксперименты с точностью.....	9
Эксперименты с вещественной частью.....	11
Задание 4.....	17
1. Введение в метод Ньютона.....	17
2. Уравнение для решения.....	17
3. Цель.....	17
4. Шаги по созданию фрактала кубического уравнения.....	17
Шаг 1: Определение сетки на комплексной плоскости.....	17
Шаг 2: Применение метода Ньютона.....	17
Шаг 3: Проверка сходимости.....	18
Шаг 4: Обработка точек, которые не сходятся.....	18
Шаг 5: Построение фрактала.....	18
5. Исходный код на Python для построения фрактала.....	19
График построенный программой.....	19
Реализация программы на Python.....	20

Задание 1

Визуализация к заданию: <https://www.desmos.com/calculator/rqg8tuecqh>

В задании требуется привести доказательства свойств множества Мандельброта.

Доказательство свойств множества Мандельброта

Лемма:

Пусть $c \in C$, $z_0 = z'_0 = 0$, $z_{n+1} = z_n^2 + c$, $z'_{n+1} = z_n'^2 + \bar{c}$.

Тогда для всех n выполняется $\overline{z_n} = z'_n$.

Доказательство по индукции:

1. База индукции: $z_0 = 0$

Тогда $(z_0)' = \bar{z}_0 = 0$, что подтверждает базовый случай.

2. Индукционное предположение:

Предположим, что для некоторого k выполнено $z'_k = \bar{z}_k$.

3. Переход: Рассмотрим следующую итерацию $k + 1$:

$$z'_{k+1} = (z'_k)^2 + \bar{c} = \bar{z}_k^2 + \bar{c} = \overline{z_k^2 + c} = \overline{z_{k+1}} = \bar{z}_{k+1}$$

Таким образом, по индукции выполняется $\bar{z}_n = z'_n$.

■

Теорема: $C' = C$

Доказательство:

- $C' = \{d : d = c', c \in C\}$
- Рассмотрим c и $\bar{c}$ и соответствующие им последовательности $\{z_n\}$ и $\{z'_n\} = \{\bar{z}_n\}$ (по лемме)
- Для любого n выполнено:

$$|z_n| = |\bar{z}_n|$$

- Если $c \in C$, то $\{z_n\}$ ограничена, следовательно, $\{\bar{z}_n\}$ также ограничена, а значит, $\bar{c} \in C$, $\bar{\bar{c}} = c \in C'$. Следовательно, $C' \subseteq C$, $C \subseteq C' \Rightarrow C' = C$.

■

Свойство, связанное с ростом $|z|$ при $|z| > 2$

- Пусть $z_{n+1} = z_n^2 + c$.

$$|z_{n+1}| = |z_n^2 + c| = |z_n \left(z_n + \frac{c}{z_n} \right)| = |z_n| \cdot \left| z_n + \frac{c}{z_n} \right|.$$

Так как $|z_n + \frac{c}{z_n}| \geq |z_n| - \frac{|c|}{|z_n|}$, имеем:

$$|z_{n+1}| \geq |z_n| \left(|z_n| - \frac{|c|}{|z_n|} \right)$$

- Докажем по индукции, что, если $z_0 = 0$, то $\forall n \in \mathbb{N} : |c| \leq |z_n|$.

3. База индукции:

$$z_1 = c \rightarrow |z_1| = |c|.$$

- Индукционное предположение:** Пусть для некоторого n выполняется $|c| \leq |z_n|$.

- Переход:** Пусть $|z_n| = 2 + \varepsilon_n$, $\frac{|c|}{|z_n|} = 1 - q_n$.

Тогда:

$$\left| z_n + \frac{c}{z_n} \right| \geq |z_n| - \frac{|c|}{|z_n|} = 1 + \varepsilon_n + q_n \geq 1 + \varepsilon_n$$

$$|z_{n+1}| \geq |z_n| \left(|z_n| - \frac{|c|}{|z_n|} \right) = (2 + \varepsilon_n)(2 + \varepsilon_n - 1 + q_n) \geq (2 + \varepsilon_n)(1 + \varepsilon_n)$$

$$|z_{n+1}| - |z_n| \geq (2 + \varepsilon_n)(1 + \varepsilon_n) - (2 + \varepsilon_n) = \varepsilon_n(2 + \varepsilon_n) > 0$$

Следовательно, $|z_{n+1}| > |z_n|$.

Следовательно, $|c| \leq |z_{n+1}|$.

6. $|z_{n+1}| = 2 + \varepsilon_{n+1} > 2 + \varepsilon_n = |z_{n+1}| \Rightarrow \varepsilon_{n+1} > \varepsilon_n$

Так как $\varepsilon_1 = c - 2 > 0$, последовательность $\{\varepsilon_n(2 + \varepsilon_n)\}$ возрастает.

Значит, $|z_n| > (n - 1)\varepsilon_1(2 + \varepsilon_1)|z_1|$.

Для любого $Z \in (0, +\infty)$ найдётся $n \in \mathbb{N}$: $|z_n|(n - 1)\varepsilon_1(2 + \varepsilon_1)|z_1| > Z$.

Следовательно, последовательность неограничена.

Задание 2

1. Введение в множество Мандельброта

Рассмотрим последовательность комплексных чисел, заданную следующим образом:

$$z_{n+1} = z_n^2 + c, \quad z_0 = 0 \quad (1)$$

Определение (Понятие множества Мандельброта)

Множество всех $c \in \mathbb{C}$, при которых последовательность z_n остается ограниченной, называется множеством Мандельброта.

Свойства множества Мандельброта:

1. Множество Мандельброта переходит само в себя при сопряжении. Иными словами, оно симметрично относительно вещественной оси;
2. Если $|c| > 2$, то c не принадлежит множеству Мандельброта;

2. Алгоритм построения

Опишем алгоритм построения множества Мандельброта:

1. Берется ограниченная часть комплексной плоскости, разбивается равномерной сеткой. Узлы сетки будут отвечать пикселям визуализируемого изображения. Каждый узел сетки будет отвечать некоторой (своей) точке c .
2. Итеративным путём строится последовательность z_n . Понятно, что итерировать до бесконечности не получится. Введем ограничение на число итераций M , а также условие «убегания» точки: модуль z_n не должен превосходить 2 (следствие свойства 2).
3. В случае, если за M итераций не произошло «убегания», считаем, что c и $\bar{c}$ (свойство 1) принадлежат множеству Мандельброта, а соответствующие пиксели закрашиваем.
4. Для получения красочных картинок можно окрашивать пиксели в какой-нибудь из цветов выбранного вами градиента в зависимости от того, на какой итерации произошла остановка.

3. Код программы

```
import numpy as np
import matplotlib.pyplot as plt

# Устанавливаем границы для осей комплексной плоскости
pmin, pmax, qmin, qmax = -2.5, 1.5, -2, 2
# Пусть  $c = p + iq$ , где  $p$  меняется в диапазоне от  $pmin$  до  $pmax$ ,
# а  $q$  меняется в диапазоне от  $qmin$  до  $qmax$ 
ppoints, qpoints = 1000, 1000 # Число точек по горизонтали и вертикали
max_iterations = 200 # Максимальное количество итераций
infinity_border = 2 # Если ушли на это расстояние, считаем, что ушли на
бесконечность

def mandelbrot(pmin, pmax, ppoints, qmin, qmax, qpoints, max_iterations,
infinity_border=2):
    # Создаем пустое изображение
    image = np.zeros((ppoints, qpoints))

    # Генерируем сетку комплексных чисел
    p, q = np.mgrid[pmin:pmax:(ppoints * 1j), qmin:qmax:(qpoints * 1j)]
    c = p + 1j * q # Комплексные числа

    # Инициализируем значения  $z$  в 0
    z = np.zeros_like(c)

    for k in range(max_iterations): # Основной цикл по итерациям
        z = z ** 2 + c # Обновляем значения  $z$  по формуле
        # Создаем маску для точек, которые "ушли" за пределы
        mask = (np.abs(z) > infinity_border) & (image == 0)
        image[mask] = k # Заполняем изображение номером итерации
        z[mask] = np.nan # Убираем точки, которые уже не будут обновляться

    return -image.T # Транспонируем изображение для правильной ориентации

# Генерируем изображение множества Мандельброта
image = mandelbrot(pmin, pmax, ppoints, qmin, qmax, qpoints)

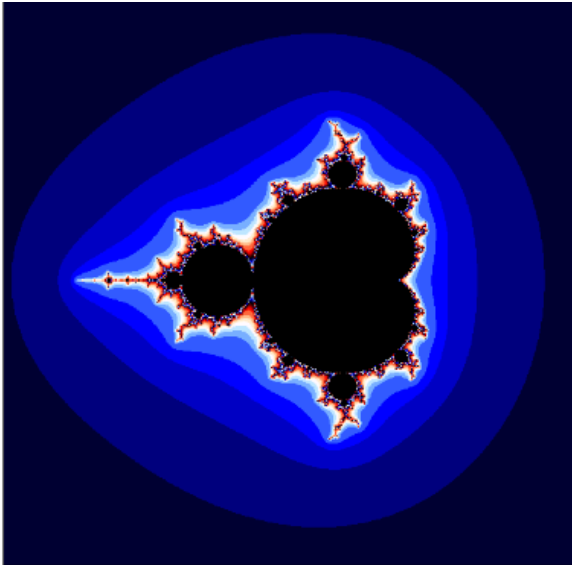
# Настраиваем отображение
plt.xticks([]) # Убираем метки по оси X
plt.yticks([]) # Убираем метки по оси Y
plt.imshow(image, cmap='flag', interpolation='none') # Отображаем изображение
plt.show() # Показать изображение
```

Для построения графического изображения множества Мандельброта можно использовать алгоритм, называемый escape-time. Суть его такова:

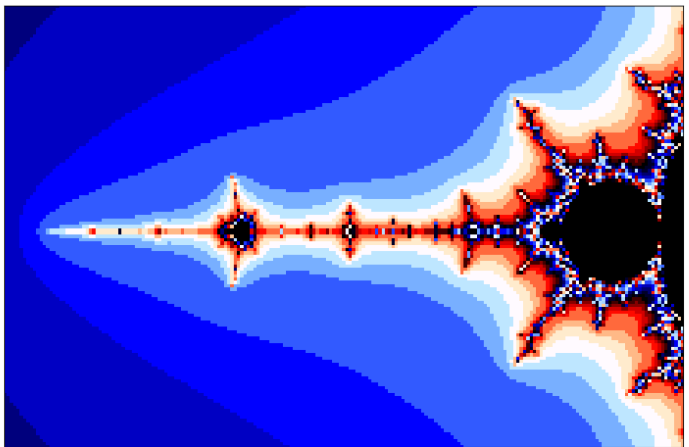
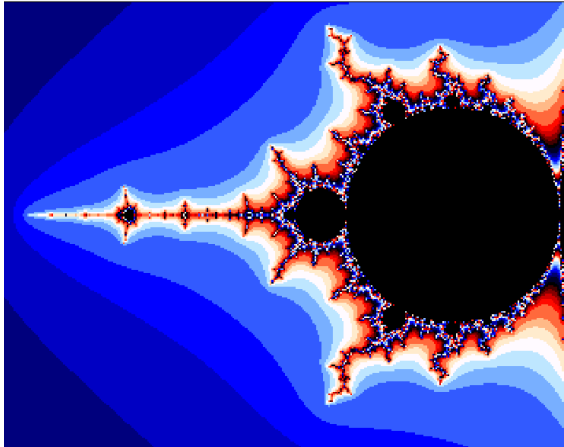
Доказано, что всё множество целиком расположено внутри круга радиуса 2 на плоскости. Поэтому будем считать, что если для точки с последовательность итераций функции (1) после некоторого большого их числа N (скажем, 200) не вышла за пределы этого круга, то точка принадлежит множеству и красится в цвет. Соответственно, если на каком-то этапе, меньшем N , элемент последовательности по модулю стал больше 2, то точка множеству не принадлежит и остается белой. Каждую точку не из множества красить в цвет, соответствующий номеру итерации, на котором ее последовательность вышла за пределы круга. Число итераций влияет на конечную визуализацию благодаря выбранной цветовой карте

4. Изображение множества при различных максимальных количествах итераций

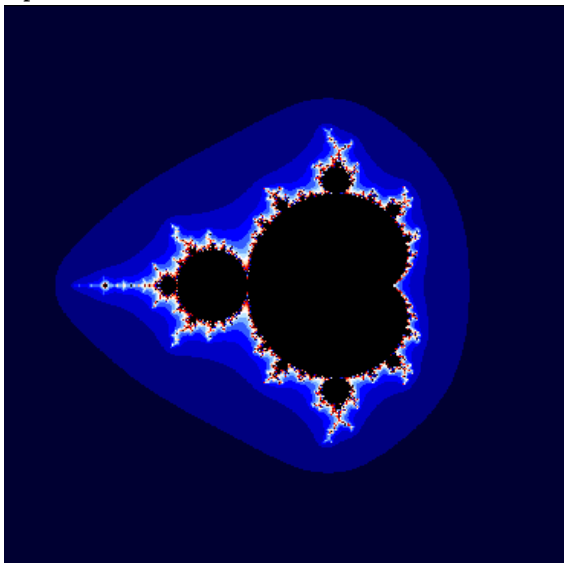
1) При $\text{max_iterations} = 300$



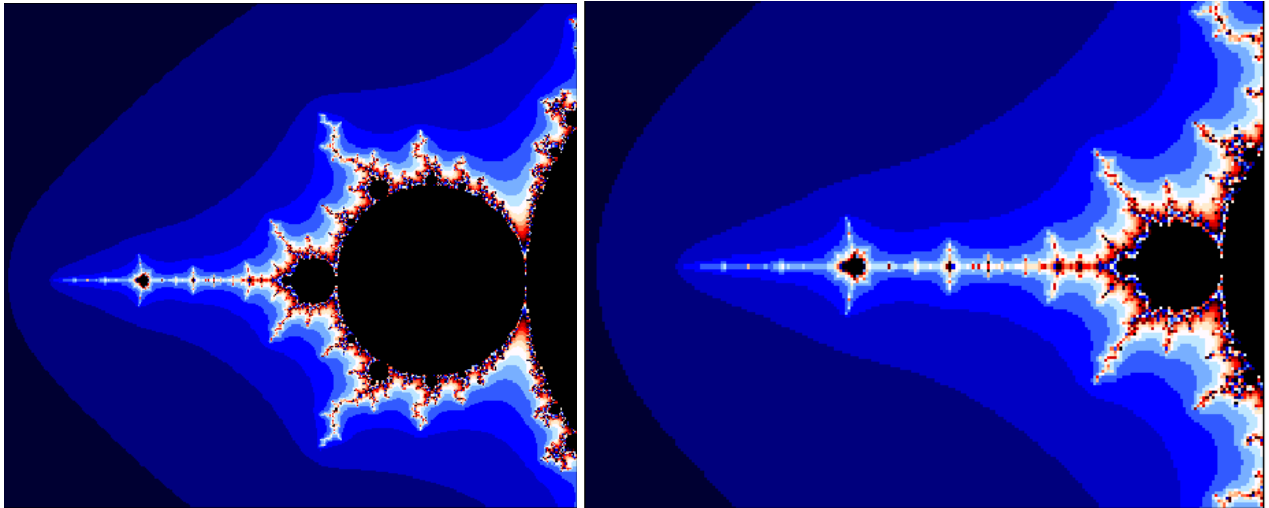
Приближенные изображения:



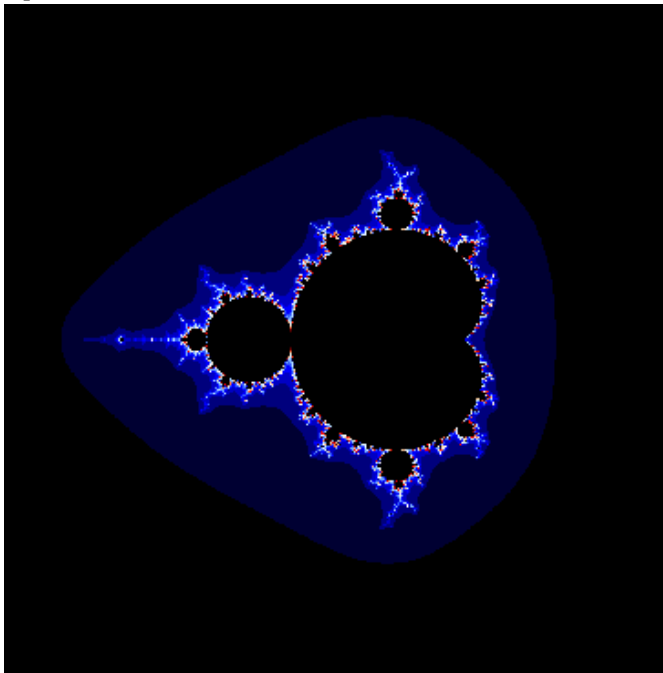
2) При $\text{max_iterations} = 500$



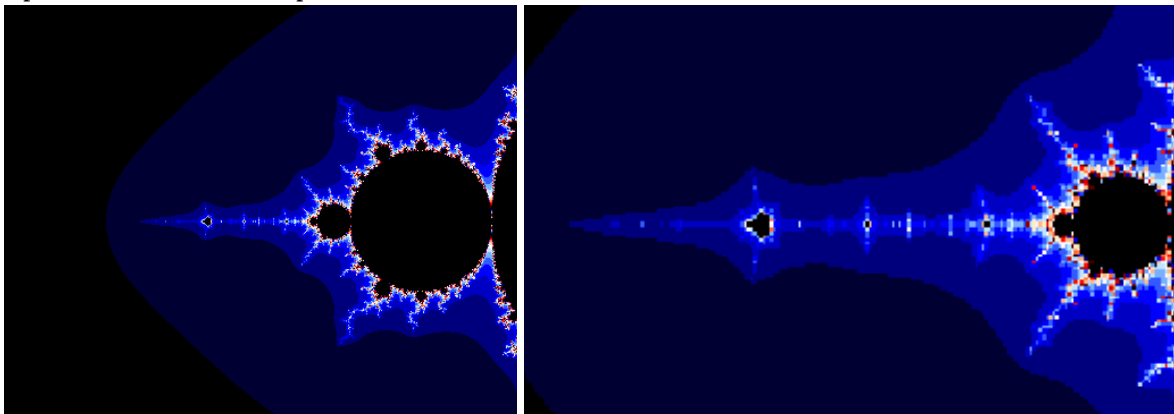
Приближенные изображения:



3) При $\text{max_iterations} = 1000$



Приближенные изображения:



Можно заметить, что при увеличении максимального количества итераций, изображение становится чуть более детализированным, особенно в областях, близких к границам множества. Можно увидеть более тонкие структуры и узоры, которые могли бы быть скрыты при меньшем числе итераций.

Задание 3

Онлайн-генератор: <https://julia-set-visualization.vercel.app>

Репозиторий: https://github.com/FeironoX5/julia_set_visualization

Множество Жюлиа строится схожим с множеством Мандельброта образом.

Код программы

```
import numpy as np
import matplotlib.pyplot as plt

IMAGE_SIZE = (800, 800)
ZOOM_FACTOR = 1
MAX_DEPTH = 100 # Максимальное количество итераций
C = complex(-1, 0.3) # Параметр множества Жюлиа

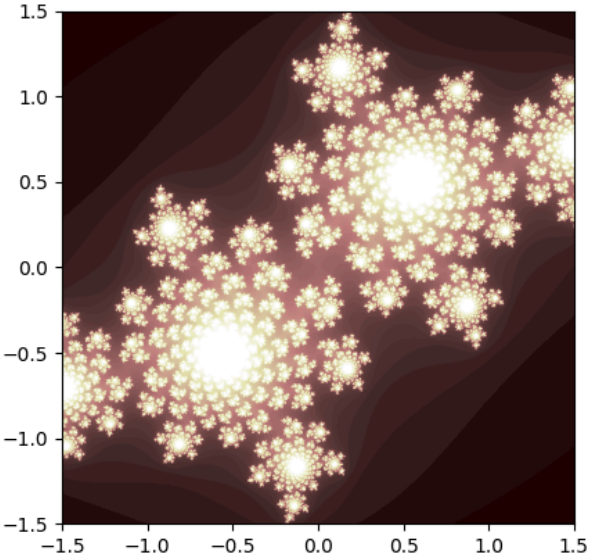
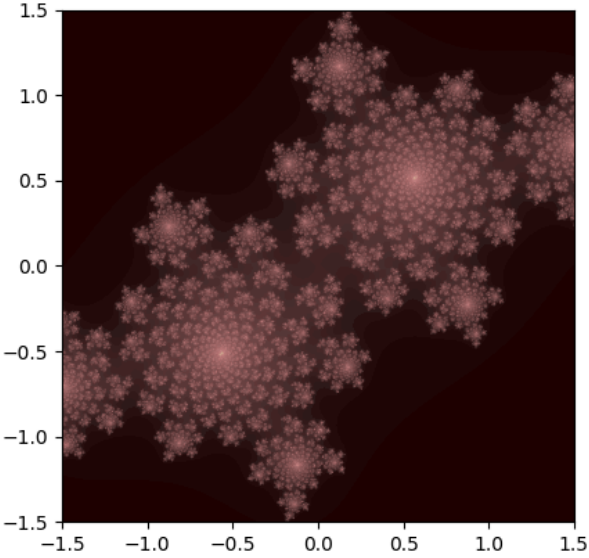
result_image = np.zeros(IMAGE_SIZE)

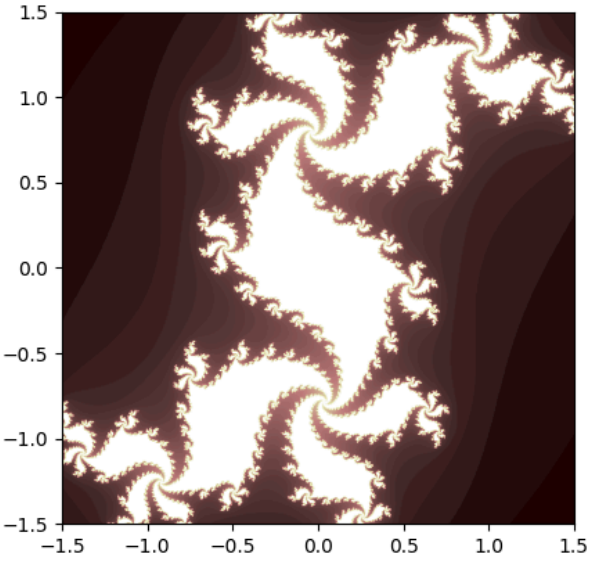
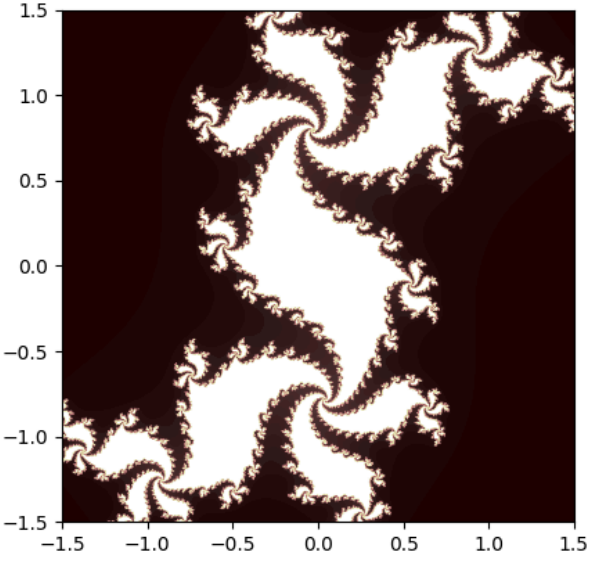
for x in range(IMAGE_SIZE[1]):
    for y in range(IMAGE_SIZE[0]):
        z_x = (x - IMAGE_SIZE[1] / 2) / (ZOOM_FACTOR * IMAGE_SIZE[1] / 2)
        z_y = (y - IMAGE_SIZE[0] / 2) / (ZOOM_FACTOR * IMAGE_SIZE[0] / 2)
        z = complex(z_x, z_y)
        for i in range(MAX_DEPTH):
            if abs(z) > 2.0:
                break
            z = z * z + C
        result_image[y, x] = i

plt.imshow(result_image, cmap='pink', extent=(- 1.5, 1.5, -1.5, 1.5))
plt.show()
```

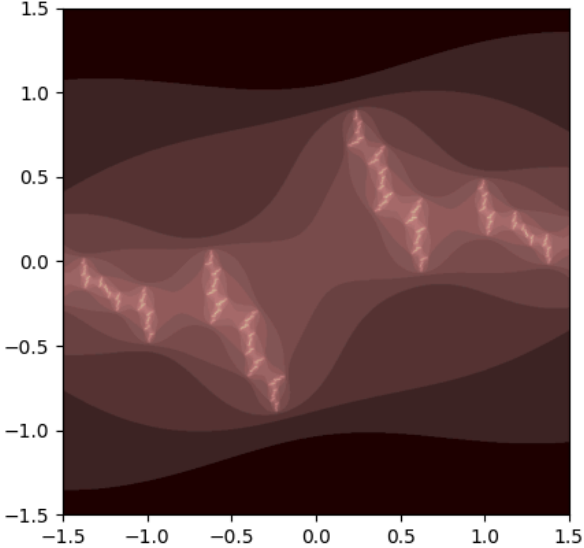
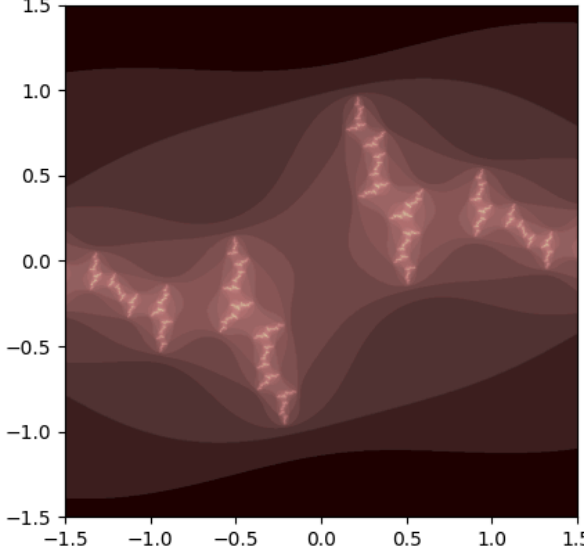
Полученные изображения

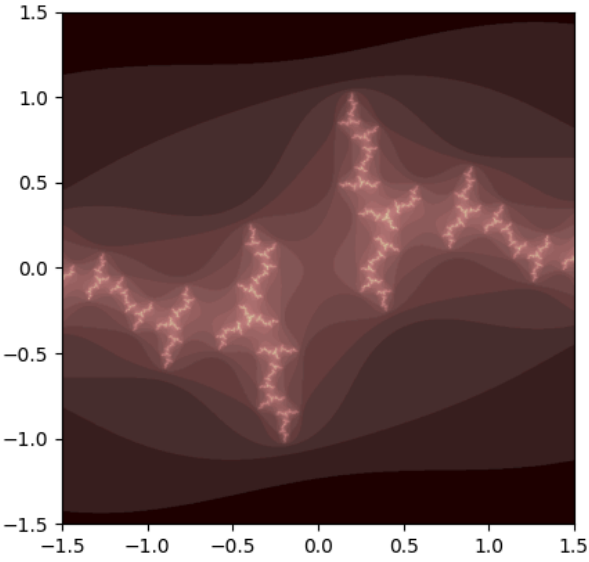
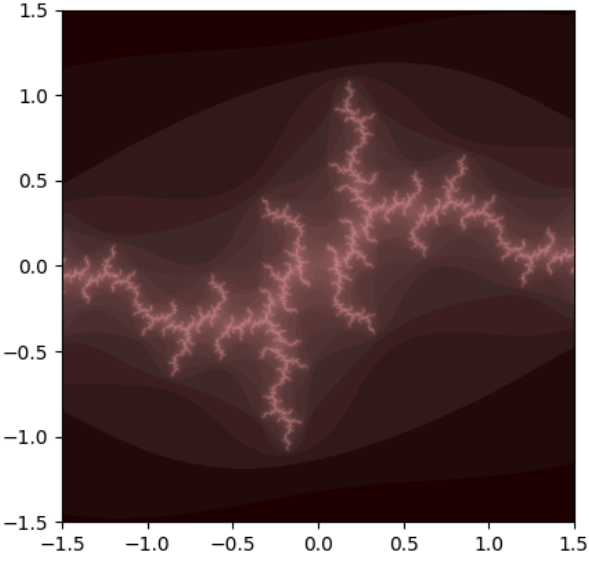
Эксперименты с точностью

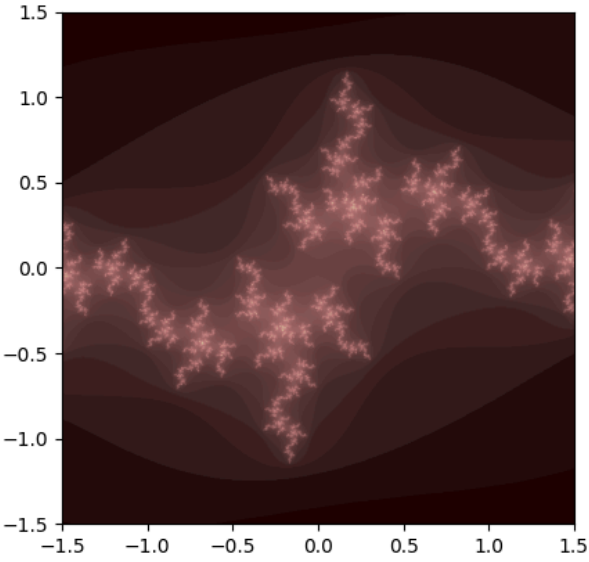
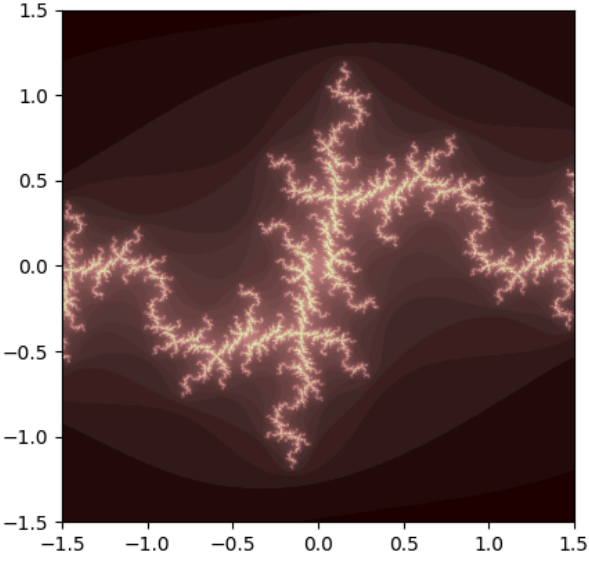
Re(C)	Im(C)	MAX_DEPTH	Картинка
0.4	0.6	100	
0.4	0.6	700	

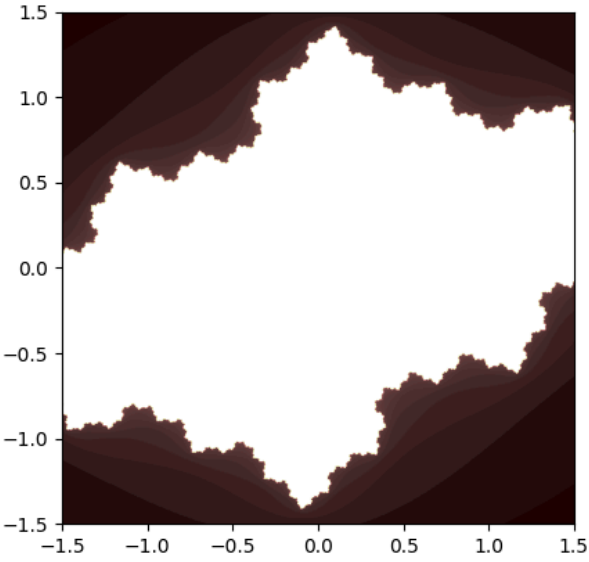
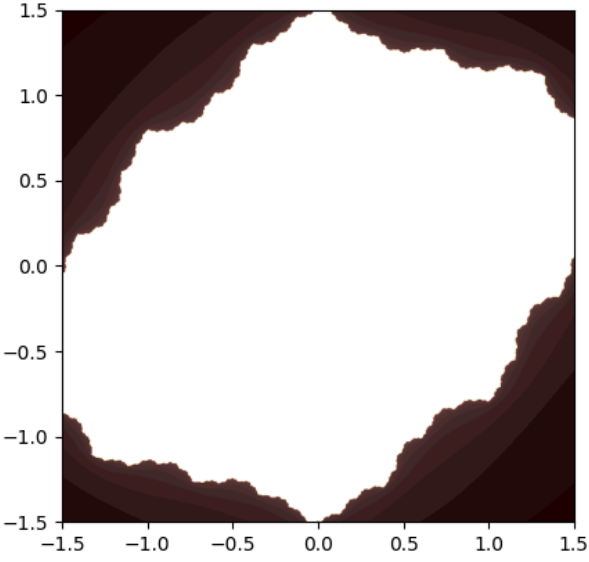
0.3	0.5	100	
0.3	0.5	700	

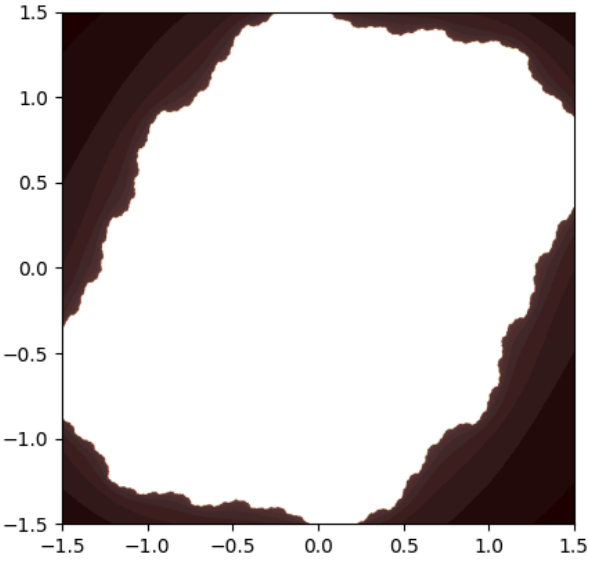
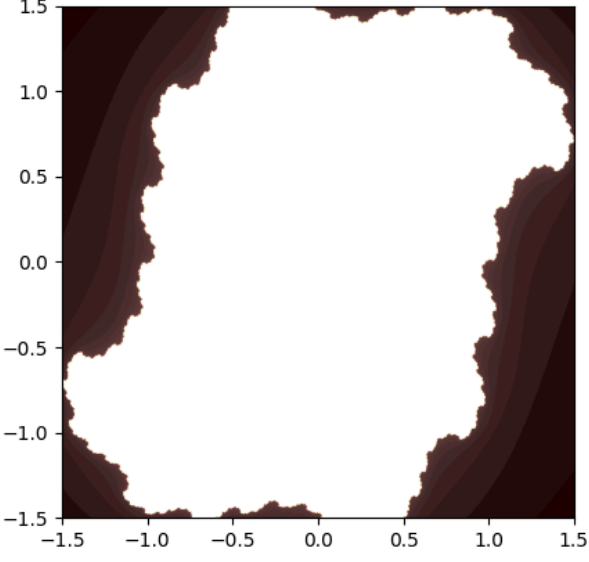
Эксперименты с вещественной частьюю

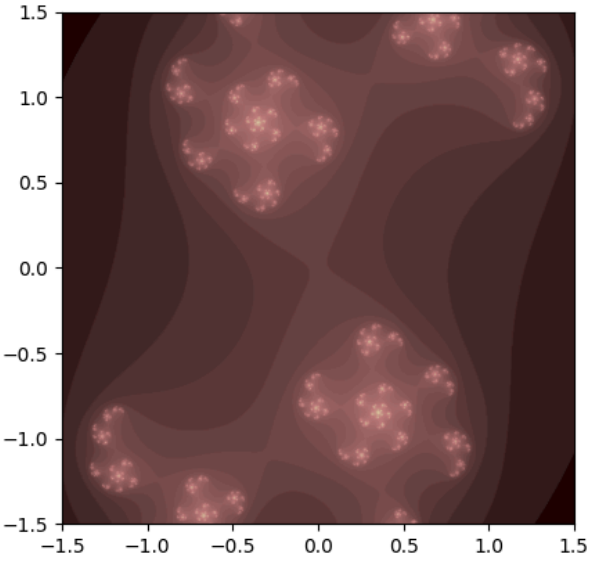
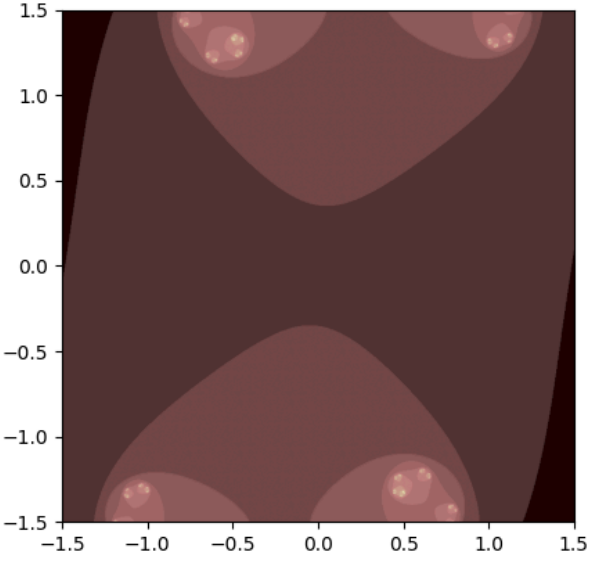
Re(C)	Im(C)	MAX_DEPTH	Картинка
-1.5	0.3	100	
-1.4	0.3	100	

-1.3	0.3	100	 A contour plot showing a function of two variables. The x and y axes both range from -1.5 to 1.5, with major ticks every 0.5 units. The plot features a complex, fractal-like boundary separating a dark red region from a lighter red region. The boundary is highly irregular and jagged, with many small-scale features. The background is a dark red color, and the boundary itself is a lighter, more vibrant red.
-1.2	0.3	100	 A contour plot showing a function of two variables. The x and y axes both range from -1.5 to 1.5, with major ticks every 0.5 units. The plot features a complex, fractal-like boundary separating a dark red region from a lighter red region. The boundary is highly irregular and jagged, with many small-scale features. The background is a dark red color, and the boundary itself is a lighter, more vibrant red.

-1.1	0.3	100	
-1	0.3	100	

-0.5	0.3	100	
-0.2	0.3	100	

0	0.3	100	
0.2	0.3	100	

0.5	0.3	100	
1	0.3	100	

Задание 4

1. Введение в метод Ньютона

Метод Ньютона — это итеративный алгоритм, который используется для нахождения корней нелинейных уравнений. Для комплексного уравнения $f(z) = 0$, метод Ньютона находит корень z , обновляя текущее значение z по следующему правилу:

$$z_{n+1} = z_n - \frac{f(z_n)}{f'(z_n)}$$

Где:

- z_n — текущее значение z .
- $f'(z_n)$ — производная функции $f(z_n)$ в точке z_n .
- $f(z_n)$ — значение функции в точке z_n .

Метод Ньютона повторяется до тех пор, пока z_n не будет сходиться к корню уравнения или пока не будет достигнуто максимальное количество итераций.

2. Уравнение для решения

Рассмотрим кубическое уравнение:

$$f(z) = z^3 - 1 = 0$$

Производная этой функции:

$$f'(z) = 3z^2$$

Это уравнение имеет **3 комплексных корня** на комплексной плоскости:

- $z_1 = 1$ (действительный корень),
- $z_2 = e^{2\pi i/3} = \frac{-1}{2} + \frac{\sqrt{3}}{2}i$
- $z_3 = e^{-2\pi i/3} = \frac{-1}{2} - \frac{\sqrt{3}}{2}i$

3. Цель

Цель — создать **фрактал Ньютона**, в котором каждая точка на комплексной плоскости будет сходиться к одному из корней уравнения. Каждая область сходимости будет окрашена в разные цвета:

- Красный для корня z_1 ,
- Зелёный для корня z_2 ,
- Синий для корня z_3 .

Границы между этими областями создадут фрактальную структуру.

4. Шаги по созданию фрактала кубического уравнения

Шаг 1: Определение сетки на комплексной плоскости

Нам нужно выбрать область на комплексной плоскости, в которой будет строиться фрактал. Это будет прямоугольник с конкретными пределами по действительной и мнимой частям:

- $\text{Re}(z)$ (действительная ось) в пределах от -2 до 2.
- $\text{Im}(z)$ (мнимая ось) в пределах от -2 до 2.

Затем мы делим эту область на сетку (аналогично пикселям на экране), где каждая точка имеет комплексные координаты z_0 .

Шаг 2: Применение метода Ньютона

Для каждой точки z_0 на сетке мы применяем метод Ньютона, чтобы вычислить последовательность значений z_n по формуле:

$$z_{n+1} = z_n - \frac{z_n^3 - 1}{3z_n^2}$$

- Начиная с z_0 , мы повторяем эту формулу, пока z_n не будет сходиться к одному из корней z_1, z_2, z_3 или пока не будет достигнуто максимальное количество итераций (обычно от 100 до 1000).

Шаг 3: Проверка сходимости

После каждой итерации проверяется, сходится ли z_n к одному из трёх корней z_1, z_2, z_3 :

- Если z_n сходится к $z_1 = 1$, мы закрашиваем эту точку в **красный** цвет.
- Если z_n сходится к $z_2 = e^{2\pi i/3}$, точка закрашивается в **зелёный** цвет.
- Если z_n сходится к $z_3 = e^{-2\pi i/3}$, точка закрашивается в **синий** цвет.

Шаг 4: Обработка точек, которые не сходятся

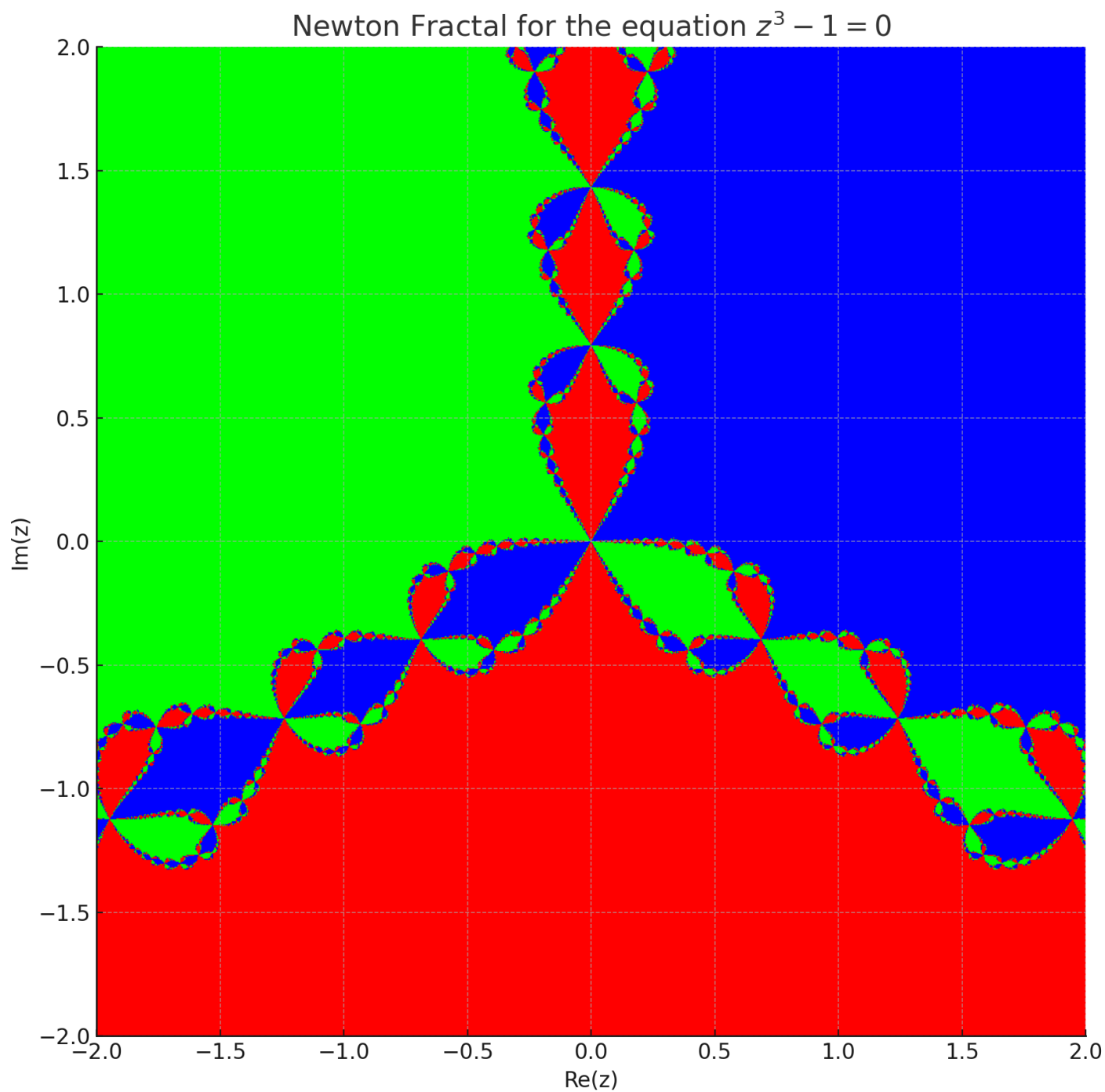
Если точка не сходится после максимального количества итераций, можно оставить её без изменений или закрасить в другой цвет (например, белый).

Шаг 5: Построение фрактала

После вычисления всех точек на сетке мы строим изображение фрактала, в котором каждая точка закрашена в соответствии с корнем, к которому она сходится. Изображение фрактала будет иметь чётко разделённые цветовые области с фрактальными границами.

5. Исходный код на Python для построения фрактала

График построенный программой



Реализация программы на Python

```
import numpy as np
import matplotlib.pyplot as plt

# Функция f(z) и её производная f'(z) для уравнения  $z^3 - 1 = 0$ 
def f(z):
    return z**3 - 1

def f_prime(z):
    return 3*z**2

# Метод Ньютона для решения комплексного уравнения  $z^3 - 1 = 0$ 
def newton_fractal(xmin, xmax, ymin, ymax, width, height, max_iter=100):
    # Создание сетки точек на комплексной плоскости
    x = np.linspace(xmin, xmax, width)
    y = np.linspace(ymin, ymax, height)
    img = np.zeros((width, height, 3))

    # Проходим по каждой точке сетки
    for i in range(width):
        for j in range(height):
            z = x[i] + 1j * y[j] # Начальная точка z_0
            for k in range(max_iter):
                z_prev = z
                z = z - f(z) / f_prime(z) # Применяем метод Ньютона
                if abs(z - z_prev) < 1e-6:
                    break

            # Определяем, к какому корню сходится точка, и закрашиваем её
            if abs(z - 1) < 1e-3:
                img[i, j] = [1, 0, 0] # Красный для корня z = 1
            elif abs(z + 0.5 + 0.866j) < 1e-3:
                img[i, j] = [0, 1, 0] # Зелёный для корня z = e^(2*pi*i/3)
            else:
                img[i, j] = [0, 0, 1] # Синий для корня z = e^(-2*pi*i/3)

    # Построение фрактала
    plt.figure(figsize=(10, 10))
    plt.imshow(img, extent=[xmin, xmax, ymin, ymax])
    plt.title("Ньютоновский бассейн для уравнения  $z^3 - 1 = 0$ ")
    plt.xlabel("Re(z)")
    plt.ylabel("Im(z)")
    plt.show()

# Вызов функции для построения фрактала Ньютона
newton_fractal(-2, 2, -2, 2, 1000, 1000)
```